



## รายงาน

OS Project Description (2569)

Concurrent Reservation System

## จัดทำโดย

- 1) นางสาวจิตาธิปไตย ลำสัน รหัสนักศึกษา 68090500408
- 2) นางสาวธนัญญา ลิ้มปิ่นทรา รหัสนักศึกษา 68090500412
- 3) นางสาวพรรณพร เหมวัตร รหัสนักศึกษา 68090500413
- 4) นายพีรภาส รุ่งวัฒนไพบูลย์ รหัสนักศึกษา 68090500415
- 5) นายณธวัค ภัทรพงศ์วัฒนา รหัสนักศึกษา 68090500450

รายวิชานี้เป็นส่วนหนึ่งของรายวิชา CSS223 Operating Systems

ภาคเรียน 1 ปีการศึกษา 2569

## คำนำ

รายงานฉบับนี้จัดทำขึ้นเป็นส่วนหนึ่งของรายวิชา CSS223 Operating Systems เพื่อศึกษาและพัฒนาระบบจองทรัพยากรแบบพร้อมกัน (Concurrent Reservation System) โดยประยุกต์ใช้แนวคิดที่สำคัญในระบบปฏิบัติการ ได้แก่ การสื่อสารกันระหว่างโพรเซส (Inter-Process Communication: IPC) การประมวลผลแบบหลายเธรด (Multithreading) การจัดการทรัพยากรร่วม (Shared Resource Management) และการควบคุมการเข้าถึงทรัพยากรร่วมด้วยกลไก Synchronization เพื่อป้องกันปัญหา Race Condition

โครงงานนี้จำลองระบบจองตัวคอนเสิร์ตบนสภาพแวดล้อม Linux/Docker โดยใช้สถาปัตยกรรมแบบ Client-Server และกลไก Message Queue สำหรับการรับส่งข้อมูลระหว่างฝั่งผู้ใช้งานและเซิร์ฟเวอร์ นอกจากนี้ยังมีการออกแบบการทดลองเพื่อเปรียบเทียบผลการทำงานในสถานการณ์ที่มีและไม่มีการใช้กลไก Synchronization เพื่อศึกษาผลกระทบที่เกิดขึ้นต่อความถูกต้องและประสิทธิภาพของระบบ

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะช่วยให้ผู้อ่านเข้าใจหลักการทำงานของ Concurrency การป้องกัน Race Condition รวมถึงแนวทางการออกแบบระบบที่รองรับการทำงานพร้อมกันได้อย่างถูกต้องและมีประสิทธิภาพ หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขอน้อมรับข้อเสนอแนะเพื่อนำไปปรับปรุงและพัฒนาต่อไป

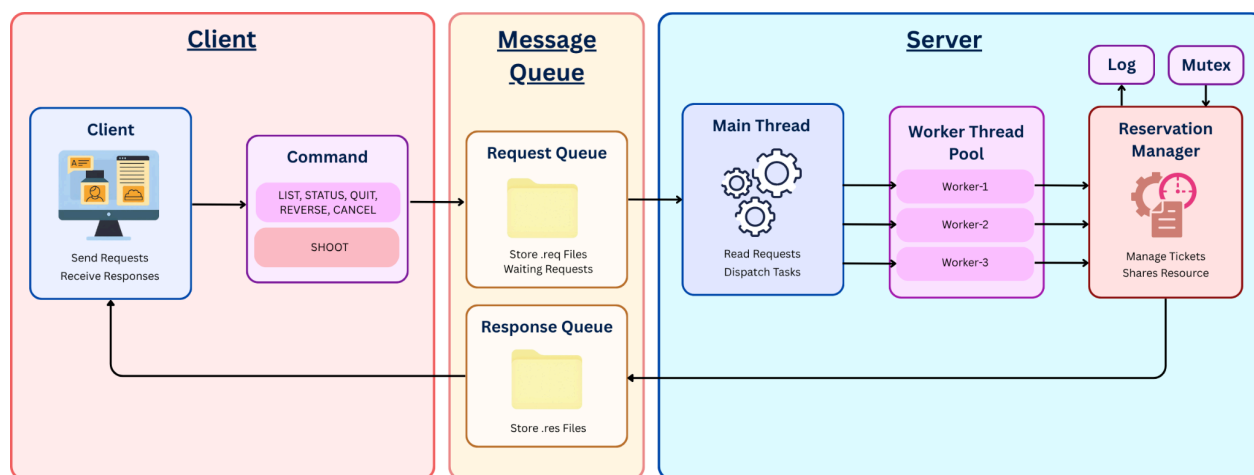
## สารบัญ

|                                           |    |
|-------------------------------------------|----|
| คำนำ                                      | ก  |
| สารบัญ                                    | ๗  |
| 1. Project Description                    | 1  |
| 2. System Architecture                    | 2  |
| 3. Message Queue Design .                 | 5  |
| 4. Message Structure                      | 10 |
| 5. Server Concurrency Model               | 12 |
| 6. Shared Resource และ Critical Section   | 16 |
| 7. สาเหตุของ Race Condition               | 17 |
| 8. บทบาทของ random delay ในการทดลอง       | 21 |
| 9. Synchronization Mechanism ที่เลือกใช้  | 23 |
| 10. ผลการทดลองทั้ง 3 กรณี                 | 26 |
| 11. ข้อจำกัดและแนวทางปรับปรุงหรือพัฒนาต่อ | 34 |

## 1. Project Description

โครงการระบบจองทรัพยากรแบบ Concurrent Reservation System เพื่อจำลองระบบจองตัวคอนเสิร์ตบนสภาพแวดล้อม Linux/Docker โดยใช้สถาปัตยกรรม Client-Server ที่สื่อสารผ่าน Message Queue ระบบประกอบด้วย Worker จำนวน 3 ตัว ทำหน้าที่ประมวลผลคำสั่งพร้อมกัน และใช้ข้อมูลตัวส่วนกลางจำนวน 50 รายการ พร้อมทั้งนำกลไก Concurrency Control (Mutex) มาควบคุมบริเวณ Critical Section เพื่อป้องกันปัญหา Race Condition เมื่อ Client หลายตัวแย่งกันจองทรัพยากร พร้อมรองรับคำสั่งพื้นฐานครบถ้วน ได้แก่ LIST, STATUS, RESERVE, CANCEL และ QUIT

## 2. System Architecture



รูปที่ 2.1: ไฟล์คำขอใน Request Queue

ระบบถูกออกแบบโดยแบ่งออกเป็น 3 ส่วนหลัก ที่ทำงานประสานกันผ่านกลไกการสื่อสารระหว่างโพรเซส (Inter-Process Communication: IPC) ดังนี้

### 2.1 ฝั่งลูกข่าย (Client)

- **Client Interface:** ทำหน้าที่เป็นส่วนติดต่อผู้ใช้ (Terminal / TUI) เพื่อรับคำสั่งจากผู้ใช้งาน (Send Requests) และรอแสดงผลการทำงานของงานที่ได้รับตอบกลับมาจาก Server (Receive Responses)
- **Command:** แบ่งกลุ่มคำสั่งออกเป็น 2 รูปแบบ:
  - **Interactive Commands:** คำสั่งการใช้งานทั่วไป ได้แก่ LIST (ดูฝั่งตัว), STATUS (ตรวจสอบสถานะตัว), RESERVE (จองตัว), CANCEL (ยกเลิกตัว), และ QUIT (ปิดโปรแกรม)
  - **Stress Test Command (SHOOT):** คำสั่งยิงคำขอจำลองโหลดขนาดใหญ่พร้อมกัน 250 คำขอ (จำลอง 5 Clients แย่งจองตัว 50 ใบพร้อมกัน) เพื่อทดสอบสภาวะ Concurrency

## 2.2 ตัวกลางสื่อสารผ่านไฟล์ (Message Queue / IPC Layer)

ทำหน้าที่เป็นช่องทางการแลกเปลี่ยนข้อมูลแบบอะซิงโครนัส (Asynchronous Decoupling) ระหว่าง Client และ Server โดยแบ่งเป็น 2 คิว:

- **Request Queue (ipc\_queue/requests/):** เก็บไฟล์คำขอ (.req) ที่ส่งมาจาก Client แต่ละตัว โดยไฟล์จะถูกจัดเรียงลำดับเวลา (Timestamp) และรันเลขคิว (Sequence) เพื่อรักษาลำดับแบบ First-In-First-Out (FIFO)
- **Response Queue (ipc\_queue/responses/):** เก็บไฟล์คำตอบ (.res) ที่ Server ประมวลผลเสร็จแล้ว โดยแยกชื่อไฟล์ตาม Client ID เพื่อให้ Client แต่ละตัวสามารถอ่านคำตอบของตนเองได้อย่างถูกต้อง

## 2.3 ฟังก์ชันแม่ข่าย (Server)

เป็นโปรเซสหลักที่ทำหน้าที่ประมวลผลคำขอแบบขนาน (Concurrency) ประกอบด้วยส่วนประกอบภายในดังนี้:

- **Main Thread (Dispatcher):** คอยสแกนอ่านคำขอจาก Request Queue และทำหน้าที่เป็นผู้กระจายงาน (Dispatch Tasks) โดยจัดสรรงานให้กับ Worker Threads
- **Worker Thread Pool:** ประกอบด้วย 3 Worker Threads (Worker-1, Worker-2, Worker-3) ทำงานคู่ขนานกันเพื่อแบ่งงานจากคิวไปประมวลผล ช่วยจำกัดการใช้งานทรัพยากรของระบบไม่ให้เกิด Thread Overload
- **Reservation Manager:** โมดูลศูนย์กลางในการจัดการสถานะตัวถัง 50 ที่นั่ง (Shared Resource) และควบคุมความถูกต้องของข้อมูล
- **Mutex (Mutual Exclusion):** กลไก Synchronization Lock (synchronized) ที่ครอบการทำงานในส่วน Critical Section เพื่อให้ Worker เข้าไปตรวจสอบและบันทึกตัวได้ทีละ 1 ตัวเท่านั้น ป้องกันปัญหาการจองซ้ำซ้อน (Race Condition / Overbooking)
- **Log (ServerLogger):** ระบบบันทึกประวัติการทำงานส่วนกลางแบบ Thread-Safe เพื่อบันทึกเวลา, หมายเลข Worker, การเข้า-ออก Critical Section, และผลลัพธ์ของแต่ละคำขอ

### 3. Message Queue Design

ระบบใช้ File-based Inter-Process Communication (IPC) เพื่อเป็นกลไกสื่อสารระหว่าง Client และ Server โดยใช้ไฟล์เป็นตัวแทนของ Message และใช้ไฟล์เตอร์เป็น Message Queue สำหรับรับส่งข้อมูลระหว่าง Process ทั้งสองฝั่ง เนื่องจาก Client และ Server ทำงานคนละ Process (คนละ JVM) จึงไม่สามารถแชร์ข้อมูลกันโดยตรงได้ การออกแบบในลักษณะนี้ช่วยให้ผู้ส่งและผู้รับสื่อสารกันผ่าน Message Queue โดยไม่จำเป็นต้องรับรู้รายละเอียดการทำงานภายในของอีกฝั่งหนึ่ง

Request และ Response ทุกฉบับถูกเก็บในรูปแบบไฟล์ข้อความ ทำให้สามารถตรวจสอบเนื้อหาของข้อความและสถานะของ Queue ได้โดยตรงจากระบบไฟล์ อีกทั้งยังสอดคล้องกับแนวคิด Message Passing Communication ที่ใช้ในระบบปฏิบัติการและระบบกระจายงาน (Distributed Systems)

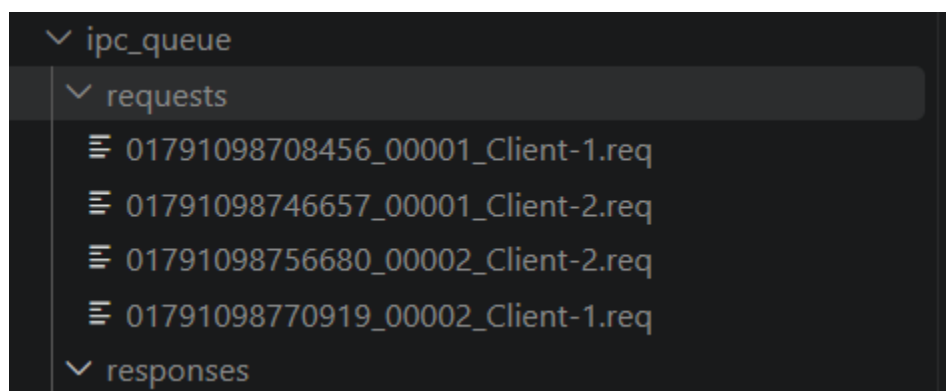
### 3.1 โครงสร้างของ Queue

ระบบประกอบด้วย Queue ทั้งหมด 3 ส่วน ได้แก่ Request Queue, Response Queue และ Internal Task Queue

| Queue               | ตำแหน่ง                 | ทิศทาง                       | หน้าที่                  |
|---------------------|-------------------------|------------------------------|--------------------------|
| Request Queue       | ipc_queue/requests/     | Client -> Server             | รับคำขอจาก Client        |
| Response Queue      | ipc_queue/responses/    | Server -> Client             | ส่งผลลัพธ์กลับให้ Client |
| Internal Task Queue | LinkedList ภายใน Server | Main Thread -> Worker Thread | เก็บคำสั่งที่รอประมวลผล  |

ตารางที่ 4.1: Queue ที่ใช้ในระบบ

Request Queue ทำหน้าที่เป็นจุดรวมของคำขอทั้งหมดที่ส่งมาจาก Client ขณะที่ Response Queue ใช้ส่งผลลัพธ์กลับไปยัง Client แต่ละรายโดยแยกไฟล์ตาม ClientID เพื่อให้แต่ละ Client ได้รับเฉพาะผลลัพธ์ของตนเอง ส่วน Internal Task Queue เป็น Queue ในหน่วยความจำของ Server ที่ Main Thread ใช้ส่งคำสั่ง RESERVE และ CANCEL ให้ Worker ประมวลผล



รูปที่ 3.1: ไฟล์คำขอใน Request Queue



### 3.2 การส่งคำขอ (Client -> Server)

Client เรียกเมธอด `sendRequest()` เพื่อเขียนคำขอลงไฟล์ชั่วคราวนามสกุล `.req.tmp` ก่อน เมื่อเขียนเสร็จสมบูรณ์จึงเปลี่ยนชื่อ (`renameTo`) เป็น `.req` เนื่องจาก Server อ่านเฉพาะไฟล์ `.req` จึงไม่มีโอกาสอ่านไฟล์ที่ยังเขียนไม่เสร็จ

```
File tmpFile = new File(REQ_DIR, filename + ".tmp");
File file = new File(REQ_DIR, filename);

try (PrintWriter out = new PrintWriter(new FileWriter(tmpFile))) {
    out.println(clientId);
    out.println(command);
    out.println(ticketId);
} catch (IOException e) {
    e.printStackTrace();
    return;
}
tmpFile.renameTo(file); // เปลี่ยนชื่อเมื่อเขียนเสร็จสมบูรณ์ ป้องกัน Server แย่งอ่าน
```

รูปที่ 3.2: การสร้าง Request File โดยเขียนข้อมูลลงไฟล์ชั่วคราว `.tmp` ก่อนเปลี่ยนชื่อเป็น `.req` เมื่อเขียนข้อมูลเสร็จสมบูรณ์

### 3.3 การรับคำขอ (Server)

Main Thread เรียกเมธอด `getIpcRequest()` โดยตรวจ Request Queue เป็นระยะ (Polling) เรียงไฟล์ `.req` ตามชื่อจากเก่าไปใหม่ แล้วเปลี่ยนชื่อไฟล์เป็น `.lock` เพื่อจองคำขอก่อนอ่าน การเปลี่ยนชื่อไฟล์เดียวกันสำเร็จได้เพียงครั้งเดียว จึงรับประกันว่าคำขอหนึ่งฉบับถูกดึงไปประมวลผลครั้งเดียว เมื่ออ่านข้อมูลครบแล้วจึงลบไฟล์ `.lock` และสร้าง Request Object หากคิดว่าง Server จะรอ 30 ms ก่อนตรวจใหม่ เพื่อไม่ให้ใช้ CPU โดยไม่จำเป็น

จากนั้น Main Thread แยกตามชนิดคำสั่ง โดย LIST, STATUS และ QUIT ตอบกลับทันที ส่วน RESERVE และ CANCEL ส่งเข้า Internal Task Queue ให้ Worker ประมวลผล

```
Arrays.sort(files, Comparator.comparing(File::getName));

for (File file : files) {
    File lockFile = new File(file.getAbsolutePath() + ".lock");

    if (file.renameTo(lockFile)) {
        String clientId = null;
        String command = null;
        int ticketId = 0;
        boolean ok = false;

        try (BufferedReader in = new BufferedReader(new FileReader(lockFile))) {
            clientId = in.readLine();
            command = in.readLine();
            ticketId = Integer.parseInt(in.readLine());
            ok = true;
        } catch (Exception e) {
            ok = false;
        }

        lockFile.delete();

        if (ok && clientId != null && command != null) {
            return new Request(command, ticketId, clientId);
        }
    }
}

return null;
```

รูปที่ 3.3: การเปลี่ยนชื่อ Request File เป็น `.lock` ก่อนอ่านข้อมูล เพื่อป้องกันการดึงคำขอเดียวกันไปประมวลผลซ้ำ

### 3.4 การส่งผลลัพธ์กลับ (Server -> Client)

Server เรียกเมธอด `sendResponse()` เพื่อเขียนผลลัพธ์ลงไฟล์ `<clientId>.res` ใน Response Queue (เขียนเป็น `.res.tmp` ก่อนแล้วเปลี่ยนชื่อ ด้วยหลักการเดียวกับการส่งคำขอ) Client ตรวจไฟล์ที่ตรงกับ ClientID ของตนทุก 30 ms เมื่อพบจึงอ่านเนื้อหาแล้วลบไฟล์ หากรอเกินเวลาที่กำหนด (10 วินาทีในโหมดปกติ และ 120 วินาทีในโหมด Stress Test) จะได้ผลลัพธ์เป็น FAILED: Request timeout

### 3.5 ข้อจำกัดของ Message Queue ในการป้องกัน Race Condition

Message Queue รับประกันเพียงการส่งและการดึงข้อความ แต่ไม่ควบคุมการเข้าถึง Shared Resource หลังดึงข้อความไปแล้ว เมื่อ Worker หลายตัวประมวลผลคำขอของตัวเองไปพร้อมกัน จึงยังเกิด Race Condition ได้

## 4. Message Structure

ระบบมี Message 2 ชนิด ได้แก่ Request Message ที่ Client ส่งไปยัง Server และ Response Message ที่ Server ตอบกลับ ทั้งสองชนิดจัดเก็บในรูปแบบข้อความ (Text-based Message) ภายในไฟล์

### 4.1 Request Message

Request Message ประกอบด้วยข้อมูล 3 ส่วน ได้แก่ ClientID, Command และ TicketID

ClientID ใช้ระบุผู้ส่งคำขอ Command ใช้ระบุประเภทการดำเนินการ เช่น LIST, STATUS, RESERVE หรือ CANCEL ส่วน TicketID ใช้ระบุหมายเลขตัวที่ต้องการดำเนินการ (1-50) โดยใช้ค่า 0 เมื่อเป็นคำสั่ง LIST ข้อมูลทั้ง 3 ส่วนเขียนเรียงกัน บรรทัดละหนึ่งค่า ดังรูปที่ 5.1

```
1 Client-1
2 RESERVE
3 2
```

รูปที่ 4.1: ตัวอย่างเนื้อหาไฟล์คำขอ

ชื่อไฟล์ของ Request ถูกออกแบบให้มีลักษณะเฉพาะเพื่อป้องกันชื่อซ้ำและจัดลำดับข้อความภายในคิว โดยประกอบด้วย timestamp 14 หลัก, sequence number 5 หลัก และ ClientID ตามรูปแบบ <timestamp>\_<sequence>\_<ClientID>.req เช่น 01712345678901\_00001\_Client-1.req Server เรียงตามชื่อไฟล์จึงได้ลำดับตามเวลาที่ส่งโดยประมาณ

เมื่อ Server อ่านไฟล์แล้ว ข้อมูลทั้ง 3 ส่วนจะถูกแปลงเป็น Request Object ซึ่งไม่มีเมฆอดสำหรับแก้ค่าหลังสร้าง จึงส่งต่อจาก Main Thread ไปยัง Worker ได้โดยไม่ต้องล็อกเพิ่ม หาก Server อ่านไฟล์ไม่ได้หรือรูปแบบข้อมูลไม่ถูกต้อง จะข้ามคำขอนั้นและลบไฟล์ทิ้ง ทำให้ Client ได้ผลลัพธ์เป็น FAILED: Request timeout เมื่อหมดเวลารอ

## 4.2 Response Message

Response Message เป็นข้อความ String ที่เขียนลงไฟล์ <clientId>.res โดยไม่แยกเป็นฟิลด์ แต่ใช้คำนำหน้าบอกผลลัพธ์ ดังตารางที่ 4.1

| คำสั่ง  | ข้อความตอบกลับ                     | ความหมาย                         |
|---------|------------------------------------|----------------------------------|
| RESERVE | SUCCESS: Reserved ticket 10        | จองสำเร็จ                        |
|         | FAILED: Ticket 10 already reserved | ตั๋วถูกจองแล้ว                   |
| CANCEL  | SUCCESS: Cancelled ticket 10       | ยกเลิกสำเร็จ                     |
|         | FAILED: Cannot cancel ticket 10    | ยกเลิกไม่ได้                     |
| STATUS  | Ticket 10 is RESERVE by Client-1   | ตั๋วถูกจองโดย Client-1           |
|         | Ticket 10 is AVAILABLE             | ตั๋วว่าง                         |
| LIST    | [Ticket 1: AVAILABLE] ...          | หลายบรรทัด บรรทัดละหนึ่งตัว      |
| หมดเวลา | FAILED: Request timeout            | Client รอผลลัพธ์เกินเวลาที่กำหนด |

ตารางที่ 4.1: รูปแบบ Response Message

ไฟล์ตอบกลับมีหนึ่งไฟล์ต่อหนึ่ง ClientID และเขียนเป็น .res.tmp ก่อนแล้วเปลี่ยนชื่อเป็น .res ด้วยหลักการเดียวกับ Request ในการทดสอบ Stress Test ฝั่ง Client ตัดสินว่าจองสำเร็จหรือไม่จากการมีคำว่า SUCCESS: ในข้อความ

## 5. Server Concurrency Model

ระบบประยุกต์ใช้รูปแบบสถาปัตยกรรมแบบ Dispatcher-Worker Pattern ร่วมกับ Producer-Consumer Model (Thread Pool) เพื่อจัดการคำขอที่เข้ามาพร้อมกันได้อย่างมีประสิทธิภาพ โดยมีรายละเอียดโครงสร้างและการแบ่งหน้าที่ดังนี้

### 5.1 ภาพรวมสถาปัตยกรรม (Architectural Overview)

เพื่อหลีกเลี่ยง Overhead จากการสร้างและทำลายเธรดซ้ำซ้อนตามแนวทาง Thread-per-request ซึ่งอาจนำไปสู่ปัญหาหน่วยความจำหมดหรือ Thread Exhaustion ภายใต้โหลดการทำงานสูง ระบบจึงเลือกใช้ Fixed Thread Pool ขนาด 3 Worker Threads ควบคู่กับ Main Thread (Dispatcher) แนวทางนี้ช่วยควบคุมการจัดสรรทรัพยากรทั้ง CPU และ Memory ให้มีประสิทธิภาพและคาดการณ์ได้ (Predictable Resource Utilization)

```
while (true) {
    Request request = MessageQueue.getIpcRequest();

    if (request != null) {
        String cmd = request.getCommand().toUpperCase();

        // LIST
        if (cmd.equals("LIST")) {
            MessageQueue.sendResponse(request.getClientId(), seatManager.listSeat().trim());
            ServerLogger.log("SERVER", "LIST", "Sent seat list to " + request.getClientId());
        }
        // STATUS
        else if (cmd.equals("STATUS")) {
            MessageQueue.sendResponse(request.getClientId(),
            seatManager.statusSeat(request.getTicketId()));
            ServerLogger.log("SERVER", "STATUS", "Sent status of ticket " + request.getTicketId()
            + " to " + request.getClientId());
        }
        // QUIT
        else if (cmd.equals("QUIT")) {
            MessageQueue.sendResponse(request.getClientId(), "Thankyou for using");
            ServerLogger.log(request.getClientId(), "QUIT", "Client finished session");
        }
        // RESERVE หรือ CANCEL
        else {
            MessageQueue.addRequest(request);
        }
    } else {
    }
}
```

รูปที่ 5.1: โค้ดการกระจายงาน (Dispatching Strategy) ของ Main Thread โดยแยกงาน Read-only กับงานแก้ไขข้อมูล

## 5.2 องค์ประกอบและบทบาทหน้าที่ (Core Components & Responsibilities)

### 5.2.1 Main Thread (Dispatcher):

ทำหน้าที่เป็นผู้เฝ้าฟังคำขอ (Listener) ที่ส่งมาจากภายนอกผ่าน Inter-Process Communication (IPC File Queue)

- การกระจายงาน (Dispatching Strategy):
  - Read-only Task: คำสั่งเช่น LIST, STATUS, และ QUIT ทาง Main Thread จะดึงข้อมูลและตอบกลับ Client โดยตรงทันที เพื่อลดภาระของคิวงานและเพิ่มความเร็วในการตอบสนอง (Low Latency)
  - State-modifying Task: คำสั่ง RESERVE และ CANCEL จะถูกส่งต่อเข้าไปยัง Internal Task Queue ของ Server เพื่อให้ Worker Threads นำไปประมวลผล

### 5.2.2 Internal Task Queue

คิวงานภายในถูกพัฒนาขึ้นด้วยโครงสร้าง LinkedList ควบคุมการเข้าถึงด้วยกลไก Monitor Object (synchronized, wait(), notifyAll())

- **Producer (Main Thread):** เมื่อมีคำขอแก้ไขข้อมูลเข้ามา เมธอด addRequest() จะบรรจุคำขอลงในคิว พร้อมเรียก notifyAll() เพื่อปลุก Worker Thread ที่หลับอยู่
- **Consumer (Worker Threads):** เรียกใช้ getRequest() เพื่อดึงงานออกจากคิว หากคิวว่าง เธรดจะเข้าสู่สถานะ Blocked / Waiting State ทันที ซึ่งช่วยขจัดปัญหา Busy Waiting (Polling) และไม่สูญเสีย CPU Cycles โดยเปล่าประโยชน์

```
public static synchronized void addRequest(Request request) {
    workerQueue.add(request);
    ServerLogger.log("QUEUE", "ADD", request.getClientId() + " -> " + request.getCommand() + " " +
        request.getTicketId());

    MessageQueue.class.notifyAll();
}

public static synchronized Request getRequest() throws InterruptedException {
    while (workerQueue.isEmpty()) {
        ServerLogger.log("QUEUE", "WAIT", "Worker waiting for request");
        MessageQueue.class.wait();
    }

    Request request = workerQueue.removeFirst();
    ServerLogger.log("QUEUE", "FETCH", request.getClientId() + " -> " + request.getCommand() + " " +
        request.getTicketId());
    return request;
}
```

รูปที่ 5.2: โค้ดการจัดการคิวงานภายใน Server โดยใช้คำสั่ง wait() และ notifyAll() เพื่อหลีกเลี่ยง Busy Waiting



### 5.2.3 Worker Thread Pool

ประกอบด้วยเธรดประมวลผล 3 ตัว (Worker-1, Worker-2, Worker-3) ที่ทำงานแบบขนานกัน แต่ละเธรดจะแข่งขันกันดึงงานจากคิว (Work Stealing/Queue Competing) และเข้าสู่กระบวนการจองหรือยกเลิกตัวผ่านโมดูล ReservationManager เมื่อประมวลผลเสร็จสิ้น Worker จะเป็นผู้สร้างและบันทึกข้อมูลตอบกลับผ่าน sendResponse() ลงใน Response Queue ของ Client แต่ละรายโดยตรง

### 5.2.4 Thread-Safe Centralized Logging

เพื่อป้องกันปัญหาข้อความแสดงผลซ้อนทับกัน (Interleaving Console Output) จากการบันทึกสถานะพร้อมกันของหลายเธรด ระบบจึงกำหนดให้เมธอด public static synchronized void log() เป็น Critical Section เพื่อรับประกันว่าข้อความบันทึกเหตุการณ์ (Audit Logs) จะถูกพิมพ์เรียงตามลำดับเวลาจริงอย่างถูกต้องและสอดคล้องกัน

## 6. Shared Resource และ Critical Section

### 6.1 Shared Resource

ระบบมีการจัดการทรัพยากรร่วมกัน ได้แก่ โครงสร้างข้อมูลตัวหรือที่นั่งจำนวน 50 ใบ ซึ่งจัดเก็บและบริหารจัดการผ่านคลาส SeatManager และประกอบด้วยออบเจกต์ Seat แต่ละใบที่ระบุ

- **หมายเลขตัว (seat\_number):** หมายเลข 1 ถึง 50
- **สถานะการจอง (status):** AVAILABLE(ว่าง) หรือ RESERVED(จองแล้ว)
- **ชื่อผู้เป็นเจ้าของ (owner):** ในรูปแบบ Client-id เช่น Client-1

ในกรณีที่ยังไม่มีใครจองสถานะการจองจะเป็น AVAILABLE(ว่าง) และชื่อผู้เป็นเจ้าของจะเป็นค่าว่าง โดยทรัพยากรเหล่านี้ถูกแชร์ให้ Worker Threads ทั้ง 3 ตัวเข้าถึงและเรียกใช้งานร่วมกันผ่านอินเทอร์เฟซ ResourceTableBridge เพื่อตรวจสอบสถานะและเปลี่ยนแปลงข้อมูลตัวตามคำสั่งที่ผู้ใช้งานส่งเข้ามาผ่าน Message Queue โดยทุก Worker จะใช้ข้อมูลตัวชุดเดียวกันร่วมกันทั้งหมด

### 6.2 Critical Section

ในระบบนี้ critical section คือบล็อกคำสั่งภายในคลาส ReservationManager ซึ่งเป็นพื้นที่ที่โปรแกรมต้องดำเนินการตรวจสอบสถานะของตัวร่วมกับการเขียนข้อมูลเปลี่ยนแปลงสถานะและชื่อผู้จอง เนื่องจากกระบวนการดังกล่าวมีการสุ่มหน่วงเวลา (randomDelay) และเปิดโอกาสให้ Worker Threads หลายตัวเข้ามาประมวลผลตัวใบเดียวกันพร้อมกัน แต่หากปล่อยให้ทำงานโดยอิสระจะเกิดปัญหา Race Condition ขึ้นได้ ระบบจึงได้นำกลไก Mutual Exclusion ผ่านการใช้ synchronized blocks และ ReentrantLock มาครอบบริเวณ Critical Section ดังกล่าว เพื่อบังคับให้ Worker แต่ละตัวต้องเข้าคิวและสลับกันเข้ามาปฏิบัติงานทีละตัวอย่างปลอดภัย ป้องกันความเสียหายของข้อมูลตัวในระบบ

## 7. สาเหตุของ Race Condition

สภาวะที่ความถูกต้องของผลลัพธ์ในระบบขึ้นอยู่กับลำดับเวลา (Timing) หรือจังหวะการสลับการทำงาน (Interleaving) ของหลายเธรดที่เข้าถึงและแก้ไขทรัพยากรร่วม (Shared Resource) พร้อมกัน โดยปราศจากการควบคุมการเข้าถึงแบบพิเศษเฉพาะตัว (Mutual Exclusion)

ในระบบจองตั๋วนี้ หากปิดการใช้งานกลไก Synchronization สาเหตุหลักที่ทำให้เกิด Race Condition สามารถวิเคราะห์ได้เป็น 3 ประเด็นสำคัญ ดังนี้

### 7.1 ข้อผิดพลาดเชิงตรรกะแบบ Check-Then-Act (Non-Atomic Operation)

ในฟังก์ชันการประมวลผลคำขอจองตั๋ว (processReservation) ประกอบด้วยการทำงาน 2 ลำดับย่อยที่ต้องพึ่งพากัน:

- **Check (อ่านสถานะ):** ตรวจสอบว่าตั๋วใบที่ต้องการยังว่างอยู่หรือไม่ ผ่านเมธอด `isAvailable(ticketId)`
- **Act (แก้ไขสถานะ):** เปลี่ยนสถานะตั๋วและบันทึกรหัสผู้ถือสิทธิ์ ผ่านเมธอด `setReserved(ticketId, clientId)`

หากไม่มีการควบคุม Critical Section ด้วย Mutex หรือ Monitor การทำงานทั้งสองขั้นตอนนี้จะไม่เป็นเนื้อเดียวกัน (Non-Atomic Operation) ส่งผลให้เกิด **Vulnerable Window** ระหว่างจังหวะการตรวจสอบสถานะกับการปรับปรุงข้อมูล ซึ่งเปิดช่องว่างให้เธรดอื่นแทรกเข้ามาอ่านข้อมูลเดิมที่ยังไม่ได้ถูกอัปเดตได้

```
private boolean processReservation(int ticketId, String clientId, int workerId) {
    ServerLogger.log("Worker-" + workerId, "LOCK", "Entering critical section");

    boolean available = (resourceTable != null) && resourceTable.isAvailable(ticketId);

    randomDelay();

    if (available) {
        if (resourceTable != null) {
            resourceTable.setReserved(ticketId, clientId);
        }
    }
}
```

รูปที่ 7.1: โค้ดส่วนที่มีความเสี่ยงต่อ Race Condition จากปัญหา Check-Then-Act โดยมีช่วง randomDelay() เป็นช่องโหว่

## 7.2 การสลับบริบทการทำงานของเธรด (Thread Interleaving & Preemption)

ภายใต้ระบบปฏิบัติการแบบ **Preemptive Multitasking** ตัวจัดสรรเวลา (CPU Scheduler) สามารถสลับการทำงาน (Context Switch) ได้ตลอดเวลา ประกอบกับการใส่ randomDelay() เพื่อจำลองความหน่วงของ I/O หรือการประมวลผล ยิ่งขยายช่วง Vulnerable Window ให้กว้างขึ้น จนเกิดลำดับเหตุการณ์ที่ผิดพลาด ดังตารางลำดับเวลาต่อไปนี้:

| ลำดับเวลา<br>(T) | Worker-1<br>(Client-A)                     | Worker-2<br>(Client-B)                       | สถานะตัว           |
|------------------|--------------------------------------------|----------------------------------------------|--------------------|
| T1               | ตัวว่าง                                    | (รอในคิว)                                    | AVAILABLE          |
| T2               | randomDelay() (ยังไม่ทันบันทึก)            | ตัวว่าง                                      | AVAILABLE          |
| T3               | randomDelay()                              | randomDelay()                                | AVAILABLE          |
| T4               | สิ้นสุด Delay บันทึกว่า Client-A จองสำเร็จ | randomDelay()                                | RESERVE (Client-A) |
| T5               | แจ้ง Client-A: SUCCESS                     | สิ้นสุด Delay บันทึกพบว่า Client-B จองสำเร็จ | RESERVE (Client-B) |
| T6               | (เสร็จสิ้น)                                | แจ้ง Client-B: SUCCESS                       | RESERVE (Client-B) |

ตารางที่ 7.2: ตารางแสดงลำดับช่วงเวลาเปรียบเทียบกรณีที่เกิด Race Condition

### 7.3 ผลกระทบและความผิดปกติของระบบ (System Anomalies)

จากเหตุการณ์ข้างต้น ก่อให้เกิดความเสียหายเชิงข้อมูลและความขัดแย้งของสถานะระบบ (State Inconsistency) 3 ประการ:

- **การจองซ้ำซ้อน (Double Booking / Overbooking):** ตัวใบเดียวกันถูกยืนยันการจองสำเร็จให้แก่ Client ทั้งสองรายพร้อมกัน ซึ่งละเมิดเงื่อนไขความถูกต้อง (Correctness Invariant) ของระบบจองตั๋ว
- **ข้อมูลสูญหายจากการเขียนทับ (Lost Update):** การบันทึกสถานะของ Client-A ที่เวลา T4 ถูก Worker-2 เขียนทับด้วยสิทธิ์ของ Client-B ที่เวลา T5 โดยสมบูรณ์ ส่งผลให้สิทธิ์ของ Client-A หายไปจากระบบโดยไม่มีการแจ้งเตือนความล้มเหลว
- **ความไม่สอดคล้องของสิทธิ์ครอบครอง (Inconsistent Ownership):** เมื่อ Client-A พยายามส่งคำสั่งยกเลิกตั๋ว (CANCEL 10) ในภายหลัง ระบบจะปฏิเสธคำขอเนื่องจากสิทธิ์ปัจจุบันในฐานะข้อมูลกลายเป็นของ Client-B ส่งผลให้ Client-A ไม่สามารถจัดการตั๋วที่ตนเองได้รับข้อความยืนยันสำเร็จได้

## 8. บทบาทของ random delay ในการทดลอง

### 8.1 บทบาทการทำงาน

ในสถานการณ์การทำงานจริง ขั้นตอนตรวจสอบและขั้นตอนเปลี่ยนสถานะมักเกิดขึ้นอย่างรวดเร็วมากในระดับไมโครวินาที ทำให้ช่วงเวลาที่เสี่ยงต่อการเกิดปัญหา หรือที่เรียกว่า race window นั้นแคบมาก แม้โค้ดจะมีข้อบกพร่องด้าน concurrency อยู่จริง ก็มีโอกาสน้อยมากที่สอง thread จะเข้ามาทำงานชนกันพอดีในช่วงเวลาสั้นๆ นั้น ส่งผลให้การรันโปรแกรมหลายครั้งอาจไม่แสดงปัญหาให้เห็นเลย ทำให้ยากต่อการพิสูจน์และนำไปอธิบายในรายงาน การหน่วงเวลาแบบสุ่มที่ใส่เพิ่มเข้าไปจึงทำหน้าที่ขยาย race window ให้กว้างขึ้นอย่างมีนัยสำคัญ เปิดโอกาสให้ thread อื่นมีเวลาเพียงพอที่จะเข้ามาทำขั้นตอนตรวจสอบในช่วงเวลาเดียวกันได้ทัน ทำให้ race condition เกิดขึ้นให้เห็นได้จริงเมื่อทดสอบร่วมกับพีเจอรียิ่งค่าของจำนวนมากพร้อมกัน อย่าง SHOOT

## 8.2 เหตุผลที่เลือกใช้ค่าน่วงแบบสุ่ม

เหตุผลที่เลือกใช้ค่าน่วงแบบสุ่มแทนที่จะเป็นค่าคงที่ตายตัวก็เพื่อให้ผลการทดลองใกล้เคียงกับสภาพการทำงานจริงมากขึ้น หากน่วงด้วยค่าคงที่เท่ากันทุกครั้ง thread ทั้งหมดจะเคลื่อนที่ไปพร้อมกันเป็นจังหวะเดียวกันตลอด ซึ่งไม่สะท้อนความเป็นจริง ในขณะที่การสุ่มค่าน่วงทำให้แต่ละ thread มาถึงขั้นตอนเปลี่ยนสถานะในเวลาที่แตกต่างกันเล็กน้อย เลียนแบบความหน่วงของเครือข่ายและเวลาในการประมวลผลที่เกิดขึ้นจริง ซึ่งสอดคล้องกับการจำลอง network jitter ที่ฝั่ง Client ก็มีการใส่ไว้เช่นกัน ในพีเจอร์ SHOOT

```
private void randomDelay() {
    try {
        int delay = (int)(Math.random() * 450) + 50; // 50-500 ms
        Thread.sleep(delay);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
```

รูปที่ 8.2: โค้ดเมธอด randomDelay() ที่หน่วงเวลาแบบสุ่มระหว่าง 50 ถึง 500 มิลลิวินาที แทรกอยู่ระหว่างขั้นตอนตรวจสอบสถานะ (check) และขั้นตอนเปลี่ยนสถานะที่นี้ (act) เพื่อขยายช่วง race window ให้เห็นปัญหา race condition ได้ชัดเจนขึ้นระหว่างการทดลอง



## 9. Synchronization Mechanism ที่เลือกใช้

### 9.1 synchronized keyword (Intrinsic Lock / Monitor Lock)

หลักการทำงานคือ synchronized (this) ใช้ ตัว ReservationManager instance เอง เป็น lock object (monitor) เมื่อ worker thread หนึ่งเข้าไปใน block นี้ได้ จะถือ lock ของ instance นั้นไว้ ทำให้ worker thread อื่น (ที่ใช้ reservationManager ตัวเดียวกัน ตามที่เห็นใน Worker.java) ต้องรอนจนกว่า thread แรกจะออกจาก block (ปลดล็อกอัตโนมัติ) ก่อนถึงจะเข้าไปทำงานต่อได้ ทำให้ขั้นตอน check (isAvailable) และ act (setReserved) ถูกรวมเป็นหนึ่งหน่วยที่แบ่งแยกไม่ได้ (atomic) เมื่อมองจากมุมของ thread อื่น เมธอด cancel() ก็ใช้หลักการเดียวกัน ล็อกที่ instance เดียวกันกับ reserve() ซึ่งถูกต้อง เพราะทั้งสองเมธอดแก้ไขข้อมูลตัวชุดเดียวกัน ถ้าล็อกคนละตัวกัน ก็ยังมีโอกาสเกิด race ระหว่าง reserve กับ cancel พร้อมกันอยู่ดี

```
public boolean reserve(int ticketId, String clientId, int workerId) {
    if (useSynchronization) {
        synchronized (this) {
            return processReservation(ticketId, clientId, workerId);
        }
    } else {
        return processReservation(ticketId, clientId, workerId);
    }
}
```

รูปที่ 9.1: โค้ดส่วนที่มีส่วนประกอบของการระบบเลือกใช้ synchronized block ของ Java เป็นกลไกหลักในการป้องกัน race condition ที่จุดวิกฤต (critical section) ของการจอง/ยกเลิกตั๋ว

การจองหนึ่งครั้งใน processReservation() ประกอบด้วยสามขั้นตอนเรียงกัน คือ ตรวจสอบสถานะที่นั่ง (check) หน่วงเวลาเพื่อจำลองการประมวลผล (delay) และเปลี่ยนสถานะเป็นจองแล้ว (act) หากสามขั้นตอนนี้ไม่ได้ถูกรวมให้เป็นหน่วยเดียวที่แบ่งแยกไม่ได้ (atomic) ก็เป็นไปได้ที่ Thread สองตัวจะเข้ามาทำขั้นตอน check พร้อมกัน เห็นว่าที่นั่งว่างเหมือนกันทั้งคู่ เพราะฉะนั้นยังไม่มีใครจองสำเร็จเลย แล้วทั้งคู่ก็เดินทางไปทำขั้นตอน act ทับกัน ผลคือที่นั่งใบเดียวถูกจองซ้ำโดยสองคน ซึ่งเป็นสิ่งที่ระบบต้องป้องกันไม่ให้เกิดขึ้น

นี่คือเหตุผลที่ระบบต้องมีกลไก synchronization เข้ามาควบคุมว่า ในแต่ละช่วงเวลาจะมีได้เพียง Thread เดียวเท่านั้นที่เข้าไปทำขั้นตอน check และ act ของตั๋วใบเดียวกัน

## 9.2 wait() / notifyAll()

กลไกนี้ต่างจากการล็อกใน ReservationManager ตรงที่ไม่ได้แค่ป้องกันการเขียนซ้อนกันเฉยๆ แต่ยังแก้ปัญหาการใช้ CPU โดยเปล่าประโยชน์ด้วย หาก Worker ไม่มีงานให้ทำ แทนที่จะวน loop เช็คคิวซ้ำ ๆ ตลอดเวลา (busy-waiting ซึ่งกิน CPU สูงโดยเปล่าประโยชน์) wait() จะทำให้ Thread นั้นเข้าสู่สถานะหลับรออยู่เฉยๆ โดยไม่กิน CPU และปลดล็อกชั่วคราวให้ Thread อื่นทำงานต่อได้ เมื่อมีคำขอใหม่เข้ามาผ่าน addRequest() เมธอด notifyAll() จะปลุก Worker ทุกตัวที่กำลังรออยู่ให้ตื่นขึ้นมาพร้อมกัน จากนั้นแต่ละตัวจะแย่งกันขอ lock อีกครั้งเพื่อเข้าไปหยิบงานจากคิว ตัวที่ได้ lock ก่อนก็จะหยิบงานไปก่อน ส่วนที่เหลือจะพบว่าคิวว่างแล้วก็กลับไป wait() ต่อ

การใช้ while (workerQueue.isEmpty()) แทนที่จะเป็น if เป็นรายละเอียดสำคัญที่ป้องกันปัญหา spurious wakeup (กรณีที่ Thread ถูกปลุกขึ้นมาโดยไม่มีเหตุผลจริง ๆ จาก JVM) และป้องกันกรณีที่ Worker หลายตัวถูกปลุกพร้อมกันแต่มีงานในคิวเพียงชิ้นเดียว

```
public static synchronized void addRequest(Request request) {
    workerQueue.add(request);
    MessageQueue.class.notifyAll();
}

public static synchronized Request getRequest() throws InterruptedException {
    while (workerQueue.isEmpty()) {
        MessageQueue.class.wait();
    }
    return workerQueue.removeFirst();
}
```

รูปที่ 9.2: โค้ดส่วนที่มีการจัดการคิวคำขอระหว่างฝั่งที่สร้างงาน (producer) กับฝั่ง Worker ที่มาหยิบงานไปทำ (consumer) โดยใช้ synchronized เข้ากับ wait()/notifyAll()

## 10. ผลการทดลองทั้ง 3 กรณี

### 10.1 เงื่อนไขการทดลอง

ทุกกรณีใช้ stress test เดียวกัน: 250 request ส่งพร้อมกัน, ticket 50 ใบ, ticket ละ 5 client แย่งกันจอง, random delay 50–500 ms ระหว่าง check และ update ตัวแปรที่เปลี่ยนมีเพียงจำนวน worker และการใช้ Mutex ก่อนรันแต่ละกรณี รีเซ็ต server เพื่อให้ ticket กลับเป็น AVAILABLE ทั้งหมด

### 10.2 ผลการทดสอบ

Experiment 1 ) ใช้ Worker เพียง 1 ตัว

ผลการทดลองครั้งที่ 1)

```
+-- STRESS TEST RESULTS -----+
| Total Requests Sent : 250
| Successful Bookings : 50
| Rejected Bookings   : 200
| Execution Time      : 186 ms
+-----+

+- MENU -----+
| [1] LIST      - show all resources
| [2] STATUS    - check one resource
| [3] RESERVE   - reserve a resource
| [4] CANCEL    - cancel your reservation
| [5] SHOOT     - stress test (5 clients each Resource)
| [0] QUIT
| You can also type a command, e.g.  RESERVE 10
+-----+

Client-1 > |
```

ผลการทดลองครั้งที่ 2)

```
+-- STRESS TEST RESULTS -----+
| Total Requests Sent : 250
| Successful Bookings : 50
| Rejected Bookings   : 200
| Execution Time      : 215 ms
+-----+

+-- MENU -----+
| [1] LIST          - show all resources
| [2] STATUS        - check one resource
| [3] RESERVE       - reserve a resource
| [4] CANCEL        - cancel your reservation
| [5] SHOOT         - stress test (5 clients each Resource)
| [0] QUIT
| You can also type a command, e.g.  RESERVE 10
+-----+

Client-1 > |
```

ผลการทดลองครั้งที่ 3)

```
+-- STRESS TEST RESULTS -----+
| Total Requests Sent : 250
| Successful Bookings : 50
| Rejected Bookings   : 200
| Execution Time      : 175 ms
+-----+

+-- MENU -----+
| [1] LIST          - show all resources
| [2] STATUS        - check one resource
| [3] RESERVE       - reserve a resource
| [4] CANCEL        - cancel your reservation
| [5] SHOOT         - stress test (5 clients each Resource)
| [0] QUIT
| You can also type a command, e.g.  RESERVE 10
+-----+

Client-1 > |
```

จองสำเร็จ 50 รายการจาก 250 request เท่ากับจำนวน ticket พอดี ที่เหลือ 200 request ถูกปฏิเสธเพราะ ticket ถูกจองไปแล้ว ไม่เกิด Race Condition แม้มี client ส่ง request พร้อมกัน 250 ตัว เพราะ Message Queue จัดลำดับ request และมี worker เพียงตัวเดียวดึงมาประมวลผลทีละรายการ จึงไม่มีสอง request ใดอยู่ระหว่าง check กับ update พร้อมกัน ผลเหมือนกันทั้ง 3 รอบ เวลาเฉลี่ย 192 ms (ประมาณ 0.77 ms/request) เนื่องจากไม่มี random delay จึงเป็นค่า baseline ที่เร็วที่สุด

**Experiment 2** ) ใช้ Worker 20 ตัว ไม่ใช่ Mutex/Semaphore และใส่ random delay ระหว่าง check และ update

ผลการทดลองครั้งที่ 1)

```
+-- STRESS TEST RESULTS -----+
| Total Requests Sent : 250
| Successful Bookings  : 64
| Rejected Bookings   : 186
| Execution Time      : 3787 ms
+-----+

+-- MENU -----+
| [1] LIST      - show all resources
| [2] STATUS    - check one resource
| [3] RESERVE   - reserve a resource
| [4] CANCEL    - cancel your reservation
| [5] SHOOT     - stress test (5 clients each Resource)
| [0] QUIT
| You can also type a command, e.g. RESERVE 10
+-----+

Client-1 > |
```

ผลการทดลองครั้งที่ 2)

```
+-- STRESS TEST RESULTS -----+
| Total Requests Sent : 250
| Successful Bookings : 65
| Rejected Bookings   : 185
| Execution Time      : 3766 ms
+-----+

+-- MENU -----+
| [1] LIST          - show all resources
| [2] STATUS        - check one resource
| [3] RESERVE       - reserve a resource
| [4] CANCEL        - cancel your reservation
| [5] SHOOT         - stress test (5 clients each Resource)
| [0] QUIT
| You can also type a command, e.g. RESERVE 10
+-----+

Client-1 > |
```

ผลการทดลองครั้งที่ 3)

```
+-- STRESS TEST RESULTS -----+
| Total Requests Sent : 250
| Successful Bookings : 74
| Rejected Bookings   : 176
| Execution Time      : 3698 ms
+-----+

+-- MENU -----+
| [1] LIST          - show all resources
| [2] STATUS        - check one resource
| [3] RESERVE       - reserve a resource
| [4] CANCEL        - cancel your reservation
| [5] SHOOT         - stress test (5 clients each Resource)
| [0] QUIT
| You can also type a command, e.g. RESERVE 10
+-----+

Client-1 > |
```

จองสำเร็จ 64, 65 และ 74 รายการ (เฉลี่ย 67.7) จาก 250 request ซึ่งเกินจำนวน ticket จริง 50 ใบ โดยเกินเฉลี่ย 17.7 รายการ (ช่วง 14-24) แสดงว่าเกิด Race Condition ทั้ง 3 รอบ สาเหตุคือ worker หลายตัวตรวจสอบ ticket เดียวกันแล้วเห็นว่า AVAILABLE พร้อมกัน ก่อนที่ตัวใดตัวหนึ่งจะเขียนผลกลับ เพราะ random delay ขยายช่วง race window ระหว่าง check กับ update ทำให้หลาย request ได้ SUCCESS กับ ticket ใบเดียวกัน (ผลลัพธ์ไม่ผิดคงที่เพราะขึ้นกับจังหวะการสลับ thread และค่า delay ที่สุ่ม) เวลาเฉลี่ย 3,750 ms เพราะ worker 20 ตัวหน่วงเวลาพร้อมกันได้ จึงไม่ต้องรอกัน

**Experiment 3** ) ใช้ Worker 20 ตัว ใช้ Mutex/Semaphore และใส่ random delay ระหว่าง check และ update

ผลการทดลองครั้งที่ 1)

```
+-- STRESS TEST RESULTS -----+
| Total Requests Sent : 250
| Successful Bookings : 50
| Rejected Bookings   : 200
| Execution Time      : 67637 ms
+-----+

+-- MENU -----+
| [1] LIST      - show all resources
| [2] STATUS    - check one resource
| [3] RESERVE   - reserve a resource
| [4] CANCEL    - cancel your reservation
| [5] SHOOT     - stress test (5 clients each Resource)
| [0] QUIT
| You can also type a command, e.g. RESERVE 10
+-----+

Client-1 > |
```

ผลการทดลองครั้งที่ 2)

```
+-- STRESS TEST RESULTS -----+
| Total Requests Sent : 250
| Successful Bookings : 50
| Rejected Bookings   : 200
| Execution Time      : 71298 ms
+-----+

+-- MENU -----+
| [1] LIST          - show all resources
| [2] STATUS        - check one resource
| [3] RESERVE       - reserve a resource
| [4] CANCEL        - cancel your reservation
| [5] SHOOT         - stress test (5 clients each Resource)
| [0] QUIT
| You can also type a command, e.g. RESERVE 10
+-----+

Client-1 > |
```

ผลการทดลองครั้งที่ 3)

```
+-- STRESS TEST RESULTS -----+
| Total Requests Sent : 250
| Successful Bookings : 50
| Rejected Bookings   : 200
| Execution Time      : 73023 ms
+-----+

+-- MENU -----+
| [1] LIST          - show all resources
| [2] STATUS        - check one resource
| [3] RESERVE       - reserve a resource
| [4] CANCEL        - cancel your reservation
| [5] SHOOT         - stress test (5 clients each Resource)
| [0] QUIT
| You can also type a command, e.g. RESERVE 10
+-----+

Client-1 > |
```



จองสำเร็จ 50 รายการจาก 250 request เท่ากับจำนวน ticket พอดีทั้ง 3 รอบ ที่เหลือ 200 request ถูกปฏิเสธเพราะ ticket ถูกจองไปแล้ว ไม่เกิด Race Condition แม้ใช้ worker 20 ตัวและ delay ชัดเดียวกับ Experiment 2 เพราะ synchronized ทำให้ขั้นตอน check → delay → update เป็น Critical Section ที่เข้าได้ทีละ worker เท่านั้น เวลาเฉลี่ย 70,653 ms (ประมาณ 283 ms/request) ใกล้เคียงกับ delay เฉลี่ย 275 ms ซึ่งยืนยันว่า request ถูกประมวลผลแบบต่อแถวทีละรายการ แม้จะมี worker 20 ตัว ก็ไม่ช่วยให้เร็วขึ้น

#### ตารางเปรียบเทียบ

|                    | Experiment 1 | Experiment 2   | Experiment 3 |
|--------------------|--------------|----------------|--------------|
| Worker             | 1            | 20             | 20           |
| Mutex              | ปิด          | ปิด            | เปิด         |
| Random delay       | ปิด          | เปิด           | เปิด         |
| จองสำเร็จ (เฉลี่ย) | 50           | 67.7           | 50           |
| จองเกินจริง        | 0            | 17.7           | 0            |
| เวลาเฉลี่ย         | 192 ms       | 3,750 ms       | 70,653 ms    |
| ผล                 | ถูกต้อง      | Race condition | ถูกต้อง      |

### 10.3 เปรียบเทียบและสรุป

- **ความถูกต้อง:** กรณี 1 และ 3 ถูกต้อง (จองสำเร็จ 50 รายการพอดีทุกรอบ ไม่มี ticket ใดถูกจองซ้ำ) ส่วนกรณี 2 ผิด (จองสำเร็จ 64, 65 และ 74 รายการ เกินจำนวน ticket จริงเฉลี่ย 17.7 รายการ) เห็นได้ชัดว่าการมี worker หลายตัวอย่างเดียวไม่ใช่ปัญหา เพราะกรณี 3 ใช้ worker 20 ตัวเท่ากับกรณี 2 แต่ผลถูกต้อง ปัญหาคือการเข้าถึง shared data พร้อมกันโดยไม่มี synchronization
- **ประสิทธิภาพ:** กรณี 3 (เฉลี่ย 70,653 ms) ช้ากว่ากรณี 2 (เฉลี่ย 3,750 ms) ราว 19 เท่า และเมื่อหารเป็นเวลาต่อ request (~283 ms) จะใกล้เคียง delay เฉลี่ย ~275 ms ซึ่งแสดงว่า request ถูกประมวลผลต่อแถวทีละรายการ เหมือนใช้ worker เพียงตัวเดียว สรุปว่า Mutex แลกความเร็วกับความถูกต้อง และเมื่อ critical section กินเวลาเกือบทั้งหมดของงาน (เพราะล๊อคครอบทั้ง check, delay และ update) การมีหลาย worker จึงไม่เร็วกว่า worker เดียว

- **สรุป:** Message Queue ช่วยจัดลำดับ request แต่ไม่ป้องกัน Race Condition เมื่อมี worker หลายตัวเข้าถึงข้อมูลส่วนกลางพร้อมกัน ต้องใช้ Mutex ครอบ Critical Section จึงจะได้ผลถูกต้อง โดยแลกกับเวลาที่เพิ่มขึ้น แนวทางปรับปรุงคือใช้ล็อกแยกต่อ ticket เพื่อให้ request ที่จองคนละ ticket ทำงานพร้อมกันได้

## 11. ข้อจำกัดและแนวทางปรับปรุงหรือพัฒนาต่อ

### ข้อจำกัด

1. ล็อกเดี่ยวทั้งตาราง synchronized ทำให้ request ที่จองคนละ ticket ต้องรอกัน และเมื่อล็อกครบ random delay ด้วย Exp 3 จึงช้ากว่า Exp 2 ราว 19 เท่า
2. IPC แบบไฟล์ ใช้ไฟล์ในโฟลเดอร์ ipc\_queue/ แทน Message Queue ของระบบปฏิบัติการ ต้อง polling ทุก 30 ms และอ่าน/เขียนดิสก์ทุก request
3. ข้อมูลอยู่ใน RAM ถ้า Server ดับ ข้อมูลการจองหายทั้งหมด
4. LIST/STATUS ไม่ผ่านล็อก อ่านตาราง ticket โดยไม่ล็อก อาจเห็นสถานะที่กำลังเปลี่ยน
5. ไม่มีการยืนยันตัวตน clientId มาจาก argument ของผู้ใช้ จึงแอบอ้างเป็น client อื่นได้
6. การทดลองรันอย่างละ 3 รอบ และ Exp 1 ปิด delay จึงเทียบเวลากับ Exp 2, 3 ตรงๆ ไม่ได้

### แนวทางปรับปรุง

1. ใช้ล็อกแยกต่อ ticket (fine-grained locking) เพื่อให้ ticket คนละใบทำงานพร้อมกันได้
2. เปลี่ยน IPC เป็น POSIX/System V Message Queue แทน polling ไฟล์
3. บันทึกตารางลงไฟล์หรือฐานข้อมูลเพื่อกู้คืนหลังรีสตาร์ท
4. ให้ LIST/STATUS ผ่านล็อกเดียวกัน หรือใช้ ReadWriteLock
5. เพิ่มการยืนยันตัวตนของ client